Deploying StreamIMDb Connector to Vercel (or a similar platform)

This tutorial explains how to run the addon on a free serverless service like **Vercel**. It also applies, with minor differences, to Netlify, Railway, Render, and Fly.io (see the final section).

> **Note on datacenter IPs:** some sources (VixSrc, vidsrc.cc, 2embed, multiembed) block requests coming from datacenter IPs with 403. On Vercel only the sources that accept datacenter IPs work (currently **Vidlink**) and, optionally, the **relay to a home server** (`UPSTREAM_URL`). That's why the `main` branch (Vercel) uses `proxyable: false` — the Stremio client fetches the stream directly using its own residential IP.

1. Prerequisites

- A vercel.com account (the free Hobby plan is enough)
- The repository on GitHub (a fork or your own)
- A free **TMDB API key**: themoviedb.org/settings/api

2. Required files (already present on the `main` branch)

Vercel runs serverless functions, not a persistent server. Two files handle the adaptation:

`vercel.json` — routes every request to the function:

```
{
  "version": 2,
  "rewrites": [
    { "source": "/(.*)", "destination": "/api/index" }
  ]
}
```

`api/index.js` — exports the Express app:

```
'use strict';
module.exports = require('../server');
```

And at the end of `server.js` , `app.listen` only runs outside Vercel (the platform sets `process.env.VERCEL` automatically):

```
if (!process.env.VERCEL) {
  app.listen(PORT, ...);
}
module.exports = app;
```

> If you're deploying from the `Server` branch instead, make sure these three pieces exist — `main` already has all of them.

3. Create the project on Vercel

1. Go to vercel.com/new
2. **Import Git Repository** → select `stremio-addon-streamimdb`
3. Under **Branch**, pick `main` (the branch prepared for serverless)
4. Framework Preset: **Other** (don't pick Next.js etc.)
5. Build settings: leave everything at default (no build step; `npm install` runs automatically)
6. Click **Deploy**

4. Environment variables

In **Project Settings** → **Environment Variables**, add:

Variable	Value	Required?
`TMDB_API_KEY`	your TMDB key	**Yes** (IMDb → TMDB conversion)
`PROXY_SECRET`	a random string (`openssl rand -hex 32`)	Recommended
`SERVER_URL`	`https://<your-project>.vercel.app`	Recommended
`UPSTREAM_URL`	public URL of your home server (Cloudflare Tunnel)	Optional — last-resort relay
`ALERT_WEBHOOK` / `TELEGRAM_BOT_TOKEN` + `TELEGRAM_CHAT_ID`	for alerts	Optional

After changing env vars: **Deployments** → ⋯ → **Redeploy**.

5. Test it

```
# manifest
curl https://<project>.vercel.app/manifest.json

# a movie (Star Wars 1977)
curl https://<project>.vercel.app/stream/movie/tt0076759.json

# health and source diagnostics from the Vercel IP
curl https://<project>.vercel.app/health
curl https://<project>.vercel.app/diag/sources
```

If `/diag/sources` shows `looksUsable: true` for a source, it works from Vercel's datacenter.

6. Install in Stremio

Open `https://<project>.vercel.app` in your browser and click **Install in Stremio**, or paste directly into Stremio (Add-ons → paste URL):

```
https://<project>.vercel.app/manifest.json
```

7. Free-tier limits to keep in mind

- **Function timeout:** 10s on Hobby by default (configurable up to 60s in `vercel.json` via `"functions": { "api/index.js": { "maxDuration": 60 } }`). Slow resolutions (movie-web ~30s, upstream relay ~25s) can exceed the default limit — raising `maxDuration` is worth it.
- **No persistent state:** the in-memory cache (`scraper.js`) only lives while the function instance stays warm. It still works, but cache hits are less frequent than on an always-on server.
- **No Puppeteer/Chromium:** not viable on serverless (size and timeout). `main` no longer depends on it.
- **Bandwidth:** if you use `proxyable: true` (the /hls proxy on Vercel), all video traffic goes through the function — this quickly blows past the free plan's limits. Keep `proxyable: false` on Vercel.

8. Alternatives to Vercel

Platform	Differences
Railway / Render / Fly.io	Run `node server.js` as a normal server (not serverless) — you don't need `api/index.js` or `vercel.json`, just set the start command to `npm start` and the same env vars. Timeouts stop being a problem; Render's free tier sleeps after inactivity.
Netlify	Similar to Vercel but functions use a different format — would need adapting (`netlify.toml` + wrapper). Not recommended without extra work.
VPS (Hetzner, Oracle Free, etc.)	Same as the home server: `git clone`, `npm install`, PM2. But it's a datacenter IP — the same sources blocked on Vercel stay blocked.

On **any** datacenter option, the underlying limitation is the same: sources that block datacenter IPs stay blocked. The most robust combo is **Vercel (always-on frontend) + `UPSTREAM_URL` pointing at the home server** (residential IP) — which is exactly the current architecture.